2026 SPARK ACADEMY

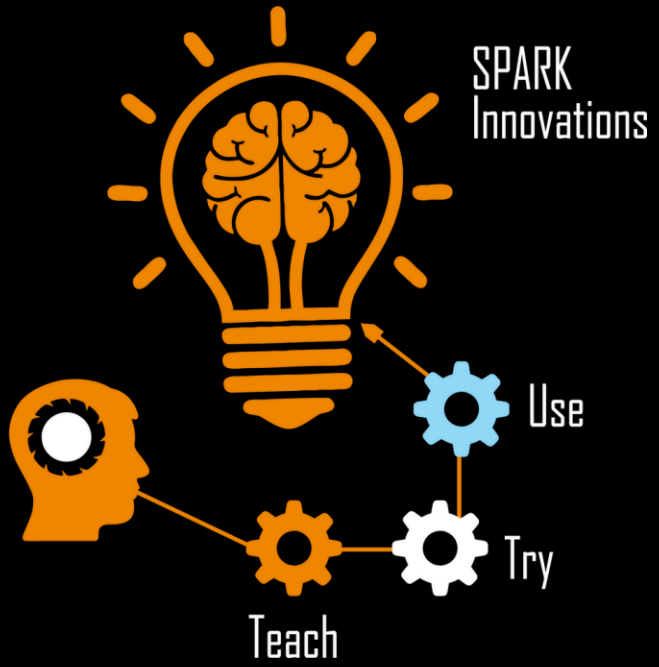
TRAIN FOR CHANGE, FROM SCIENCE TO PRACTICE

THE SPRINT AI TRAINING FOR AFRICAN MEDICAL
Imaging Knowledge Translation (SPARK) Academy

IN DEEP LEARNING & MEDICAL IMAGING

Week 4 Tutorial (Data Manipulation with Pandas)

MAR 14th 2026



Why Pandas?

The backbone of data analysis in Python

Handles Real Data

Load CSV, Excel, SQL — real hospital data formats

Explore Fast

Inspect, filter, sort thousands of patient records instantly

Clean Messy Data

Handle missing values, duplicates, inconsistent entries

Analyze & Summarize

Mean, median, counts — descriptive statistics in one line

Getting Started

Install and import Pandas

```
# Install (run once)
pip install pandas

# Import — always use 'pd' alias
import pandas as pd

# Check version
print(pd.__version__)
```

Convention

Everyone in the Python community imports pandas as pd.

You will see pd.DataFrame, pd.read_csv, etc. everywhere.

Already installed on Kaggle & Colab!

Series & DataFrames

The two core data structures in Pandas

Pandas Series

A single column of data with an index

```
import pandas as pd

# Create a Series of tumor sizes
tumor_sizes = pd.Series(
    [2.3, 1.8, 4.1, 0.9, 3.5],
    name="tumor_cm"
)
print(tumor_sizes)
```

Output:

```
0    2.3
1    1.8
2    4.1
3    0.9
4    3.5
Name: tumor_cm, dtype: float64
```

Think of it as:

Index	Value
0	2.3
1	1.8
2	4.1
3	0.9
4	3.5

Like a list, but with labels!

Pandas DataFrame

A table of data — rows and columns (like a spreadsheet)

```
patients = pd.DataFrame({
    "name": ["Fatima", "Chidi", "Amina", "Bola", "Kofi"],
    "age": [52, 34, 45, 28, 61],
    "scan_type": ["MRI", "CT", "MRI", "X-Ray", "CT"],
    "tumor_cm": [3.5, 1.2, 4.1, 0.8, 2.3],
    "diagnosis": ["Glioma", "Normal", "Meningioma", "Normal", "Glioma"]
})
print(patients)
```

Output:

```
     name  age scan_type  tumor_cm   diagnosis
0  Fatima   52       MRI       3.5       Glioma
1   Chidi   34        CT       1.2       Normal
2   Amina   45       MRI       4.1  Meningioma
3    Bola   28     X-Ray       0.8       Normal
4    Kofi   61        CT       2.3       Glioma
```

Series vs DataFrame

Series (1D)

One column of data

Like a single list with labels

Access: patients["tumor_cm"]

```
s = pd.Series([2.3, 1.8, 4.1])
print(type(s))
# <class 'pandas.core.series.Series'>
```

DataFrame (2D)

Multiple columns of data

Like a spreadsheet or table

Each column is a Series

```
df = pd.DataFrame({"a": [1,2], "b": [3,4]})
print(type(df))
# <class 'pandas.core.frame.DataFrame'>
```

Loading & Exploring Data

Getting real-world medical data into Pandas

Loading Data from CSV

pd.read_csv() — your most-used Pandas function

```
# Load a CSV file of patient scan records
df = pd.read_csv("patient_scans.csv")

# Load from a URL (Kaggle datasets, GitHub, etc.)
df = pd.read_csv("brain_tumor_data.csv")

# Other formats
df = pd.read_excel("hospital_records.xlsx")
df = pd.read_json("api_response.json")
```

Common Formats

read_csv() → .csv files
read_excel() → .xlsx files
read_json() → .json files
read_sql() → databases

CSV is the most common format in medical AI research.

First Look at Your Data

head(), tail(), shape — always start here

```
# First 5 rows
print(patients.head())

# Last 3 rows
print(patients.tail(3))

# Dimensions (rows, columns)
print(patients.shape)
```

Output: `(5, 5)` → 5 rows, 5 columns

Golden Rule

Always run these 3 commands first when loading any new dataset:

1. df.head() → see the data
2. df.shape → how big is it?
3. df.info() → column types

This takes 10 seconds and saves you hours of debugging.

Understanding Your Data

info() and dtypes — know your column types

```
# Column types and non-null counts
patients.info()
```

Output:

```
name         5 non-null    object
age          5 non-null    int64
scan_type    5 non-null    object
tumor_cm     5 non-null    float64
diagnosis    5 non-null    object
```

```
# Just the types
print(patients.dtypes)

# Column names
print(patients.columns.tolist())
```

Pandas Data Types

`int64`	Whole numbers (age, count)
`float64`	Decimals (tumor_cm, dose)
`object`	Text/strings (name, diagnosis)
`bool`	True / False
`datetime64`	Dates and times

Selecting Columns

Pull out the data you need

```
# Single column → returns a Series
names = patients["name"]
print(names)

# Multiple columns → returns a DataFrame
subset = patients[["name", "tumor_cm", "diagnosis"]]
print(subset)
```

Output:

```
     name  tumor_cm   diagnosis
0  Fatima       3.5       Glioma
1   Chidi       1.2       Normal
2   Amina       4.1  Meningioma
3    Bola       0.8       Normal
4    Kofi       2.3       Glioma
```

Key Difference

Single brackets [ ]
→ returns a Series

Double brackets [[ ]]
→ returns a DataFrame

This matters for how you can chain operations!

Selecting Rows

loc (label-based) and iloc (position-based)

```
# By position (iloc) — like list indexing
first_patient = patients.iloc[0]        # row 0
first_three = patients.iloc[0:3]        # rows 0,1,2

# By label (loc) — uses the index
patients.loc[0, "name"]                 # "Fatima"
patients.loc[0:2, ["name", "tumor_cm"]] # rows 0-2, specific cols
```

iloc — integer position

Works like Python list slicing
iloc[0:3] → excludes row 3

loc — label-based

Uses index labels, not position
loc[0:2] → includes row 2

Filtering Data

Boolean indexing — the most powerful Pandas feature

```
# Patients with tumor > 3.0 cm
critical = patients[patients["tumor_cm"] > 3.0]
print(critical)

# MRI scans only
mri_patients = patients[patients["scan_type"] == "MRI"]

# Multiple conditions (use & for AND, | for OR)
glioma_large = patients[
    (patients["diagnosis"] == "Glioma") & (patients["tumor_cm"] > 2.0)
]
```

Output:

```
   name  age scan_type  tumor_cm   diagnosis
0  Fatima   52       MRI       3.5       Glioma
2   Amina   45       MRI       4.1  Meningioma
```

Data Cleaning & Preprocessing

Handling the messy reality of hospital data

Detecting Missing Values

NaN = Not a Number — Pandas way of saying 'data is missing'

```
import numpy as np

# Create data with missing values
df = pd.DataFrame({
    "name": ["Fatima", "Chidi", "Amina", "Bola"],
    "age": [52, np.nan, 45, 28],
    "tumor_cm": [3.5, 1.2, np.nan, 0.8]
})

# Check for missing values
print(df.isnull().sum())
```

Output:

```
name        0
age         1
tumor_cm    1
dtype: int64
```

Detection Tools

isnull() → True where missing
notnull() → True where present
isnull().sum() → count missing
isnull().any() → any missing?

Always check for missing values before any analysis!

Handling Missing Values

Drop them or fill them — depends on the situation

```
# Option 1: Drop rows with any missing value
cleaned = df.dropna()

# Option 2: Drop rows where specific column is missing
cleaned = df.dropna(subset=["tumor_cm"])

# Option 3: Fill with a value
df["age"] = df["age"].fillna(df["age"].median())

# Option 4: Fill with forward/backward fill
df["tumor_cm"] = df["tumor_cm"].fillna(method="ffill")
```

When to Drop vs Fill?

Drop when:
- Few missing rows (<5%)
- Missing data is random

Fill with mean/median when:
- Numeric columns
- Cannot afford to lose rows

Fill with mode when:
- Categorical columns
- (e.g., scan_type, diagnosis)

Removing Duplicates

Same patient scanned twice? Same record entered twice?

```
# Check for duplicates
print(df.duplicated().sum())       # count of duplicate rows

# Remove exact duplicates
df_clean = df.drop_duplicates()

# Remove duplicates based on specific columns
# Keep the last entry for each patient
df_clean = df.drop_duplicates(subset=["name"], keep="last")
```

Output: 2 (2 duplicate rows found and removed)

In medical data, duplicates often happen when a patient gets re-scanned or a record is entered twice. Always check!

Renaming & Type Conversion

Fix column names and data types

```
# Rename columns (hospital data often has ugly names)
df = df.rename(columns={
    "Patient Name": "name",
    "Tumour Size (cm)": "tumor_cm",
    "Scan Date": "scan_date"
})

# Convert types
df["age"] = df["age"].astype(int)
df["scan_date"] = pd.to_datetime(df["scan_date"])
df["tumor_cm"] = pd.to_numeric(df["tumor_cm"], errors="coerce")
```

errors='coerce' → *turns invalid values into NaN instead of crashing*

Adding & Modifying Columns

Create new features from existing data

```
# Add a new column based on a condition
patients["status"] = patients["tumor_cm"].apply(
    lambda x: "Critical" if x > 3.0 else "Stable"
)

# Math on columns
patients["tumor_mm"] = patients["tumor_cm"] * 10

# String operations
patients["name_upper"] = patients["name"].str.upper()

# Categorize with multiple conditions
import numpy as np
patients["priority"] = np.where(
    patients["tumor_cm"] > 3.0, "High", "Normal"
)
```

Sorting Data

Order your data by any column

```
# Sort by tumor size (largest first)
patients_sorted = patients.sort_values("tumor_cm",
ascending=False)
print(patients_sorted)

# Sort by multiple columns
patients_sorted = patients.sort_values(
    ["diagnosis", "tumor_cm"],
    ascending=[True, False]
)
```

Output:

	name	age	scan_type	tumor_cm	diagnosis
2	Amina	45	MRI	4.1	Meningioma
0	Fatima	52	MRI	3.5	Glioma
4	Kofi	61	CT	2.3	Glioma
1	Chidi	34	CT	1.2	Normal
3	Bola	28	X-Ray	0.8	Normal

Descriptive Statistics

Summarizing your medical data

describe() — Full Summary

One line of code, complete statistical profile

```
print(patients.describe())
```

Output:

	age	tumor_cm
count	5.0	5.00
mean	44.0	2.38
std	12.9	1.35
min	28.0	0.80
25%	34.0	1.20
50%	45.0	2.30
75%	52.0	3.50
max	61.0	4.10

What each row means:

`count`	How many non-null values
`mean`	Average value
`std`	Standard deviation (spread)
`min / max`	Smallest / largest value
`25%`	25th percentile (Q1)
`50%`	Median (middle value)
`75%`	75th percentile (Q3)

Individual Statistics

Target specific calculations

```
# Central tendency
print(patients["tumor_cm"].mean())      # 2.38
print(patients["tumor_cm"].median()) # 2.3
print(patients["scan_type"].mode()[0])  # "CT" or "MRI"

# Spread
print(patients["age"].std())            # 12.94
print(patients["tumor_cm"].var())       # 1.822

# Counts
print(patients["tumor_cm"].min())       # 0.8
print(patients["tumor_cm"].max())       # 4.1
print(patients["tumor_cm"].sum())       # 11.9
```

value_counts()

```
patients["diagnosis"].value
_counts()
```

Output: Glioma 2
Normal 2
Meningioma 1

Perfect for counting categories — diagnoses, scan types, etc.

GroupBy — Split, Apply, Combine

Analyze data by category — one of Pandas' most powerful features

```
# Average tumor size by diagnosis
print(patients.groupby("diagnosis")["tumor_cm"].mean())

# Multiple stats by scan type
print(patients.groupby("scan_type").agg({
    "age": "mean",
    "tumor_cm": ["mean", "max", "count"]
}))
```

Output:

```
diagnosis
Glioma        2.90
Meningioma    4.10
Normal        1.00
Name: tumor_cm, dtype: float64
```

Clinical insight: Glioma patients average 2.9 cm tumor, Meningioma 4.1 cm, Normal 1.0 cm. groupby makes this analysis trivial.

Correlation Analysis

Are age and tumor size related?

```
# Correlation matrix (numeric columns only)
print(patients[["age", "tumor_cm"]].corr())

# Single correlation
print(patients["age"].corr(patients["tumor_cm"]))
```

Output:

```
           age  tumor_cm
age       1.00      0.72
tumor_cm  0.72      1.00
```

Reading Correlation

+1.0 → perfect positive
0.0 → no relationship
-1.0 → perfect negative

0.72 = strong positive
→ older patients tend to have larger tumors

Putting It All Together

A complete data analysis workflow

```
# Step 1: Load
df = pd.read_csv("brain_tumor_patients.csv")

# Step 2: Explore
print(df.shape)
print(df.info())
print(df.head())

# Step 3: Clean
df = df.dropna(subset=["tumor_cm"])
df = df.drop_duplicates()
df["age"] = df["age"].fillna(df["age"].median())

# Step 4: Analyze
print(df.groupby("diagnosis")["tumor_cm"].describe())
print(df["scan_type"].value_counts())

# Step 5: Filter for your research
critical = df[df["tumor_cm"] > 3.0]
print(f"Critical patients: {len(critical)}")
```

Saving Your Work

Export your cleaned data for models or sharing

```
# Save to CSV (most common)
df.to_csv("cleaned_patients.csv", index=False)

# Save to Excel
df.to_excel("cleaned_patients.xlsx", index=False)

# Save specific subset
critical = df[df["tumor_cm"] > 3.0]
critical.to_csv("critical_patients.csv", index=False)
```

index=False

Always use index=False unless you specifically need the row numbers saved.

Without it, Pandas adds an extra unnamed column of indexes to your CSV.

Common Pandas Errors

You will see these — here is how to fix them

`KeyError: 'tumor'`

Column name does not exist. Check df.columns for exact names.

`SettingWithCopyWarning`

Use .loc[] for assignment instead of chained indexing.

`ValueError: cannot convert`

Data has non-numeric values. Use pd.to_numeric(errors='coerce').

`FileNotFoundError`

Wrong file path. Use os.path.exists() to check first.

Pandas Cheat Sheet

Loading & Exploring	
Load	`pd.read_csv("file.csv")`
Shape	`df.shape`
Preview	`df.head()`
Info	`df.info()`
Columns	`df.columns.tolist()`
Select col	`df["col"]`
Select rows	`df.iloc[0:5]`
Filter	`df[df["col"] > val]`

Cleaning & Analysis	
Missing	`df.isnull().sum()`
Drop NaN	`df.dropna()`
Fill NaN	`df.fillna(value)`
Duplicates	`df.drop_duplicates()`
Sort	`df.sort_values("col")`
Group	`df.groupby("col").mean()`
Stats	`df.describe()`
Save	`df.to_csv("f.csv", index=False)`

Bonus: String Operations

Clean text columns with .str accessor

```
# Standardize text
df["diagnosis"] = df["diagnosis"].str.lower().str.strip()

# Check if column contains a substring
mri_mask = df["scan_type"].str.contains("MRI", case=False)

# Replace values
df["diagnosis"] = df["diagnosis"].str.replace("glioblastoma", "GBM")

# Extract parts
df["first_name"] = df["name"].str.split().str[0]
```

Hospital data is full of inconsistencies: 'MRI', 'mri', ' MRI ', 'Mri Brain'. Use .str methods to standardize everything before analysis. This is especially common with African hospital data where entries are often manually typed.

How to Practice

Open a Kaggle notebook

Find any medical dataset and practice: load → explore → clean → analyze

Try real datasets

Search Kaggle for 'brain tumor', 'chest xray', 'medical imaging' CSV files

Break things on purpose

Try wrong column names, bad file paths — learn what errors look like

Build a mini project

Load a dataset, clean it, compute 5 statistics, save the result. Done.

Keep Exploring Data!

Load → Explore → Clean → Analyze → Save

SPARK Academy 2026 — Train for Change, From Science to Practice